

ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



HỌC PHẦN: CÁC HỆ THỐNG THÔNG MINH
DỰ ÁN: AIRCARE

DỰ ĐOÁN CHẤT LƯỢNG KHÔNG KHÍ TRONG 1, 3, 6 GIỜ TIẾP THEO VÀ CẢNH
BÁO SỨC KHỎE CÁ NHÂN

Giảng viên hướng dẫn: TS. Mai Xuân Tráng

Lớp tín chỉ: CSE702115-1-1-25(N01)

Nhóm 8 – Lớp: KTPM(EL_2)

- | | |
|-----------------------------|----------------------|
| 1. Lê Thị Kiều Trang | MSV: 23010502 |
| 2. Quách Hữu Nam | MSV: 23012358 |
| 3. Nguyễn Minh Sang | MSV: 23010888 |
| 4. Nguyễn Đức Trường | MSV: 23010767 |

Hà Nội, tháng 10 năm 2025

THÀNH VIÊN NHÓM 8

STT	Họ và tên	Mã sinh viên	Tên tài khoản github	Nội dung công việc
1	Lê Thị Kiều Trang	23010502	agnessktt	- Làm tài liệu - Lập trình chương trình
2	Quách Hữu Nam	23012358	NamNamNN	- Thu thập thông tin - Lập trình chương trình
3	Nguyễn Minh Sang	23010888	minhsangcoder	- Thu thập thông tin - Lập trình chương trình
4	Nguyễn Đức Trường	23010767	T-Hades02	- Thu thập thông tin - Lập trình chương trình

MỤC LỤC

1. Giới thiệu	5
1.1 Bối cảnh	5
1.2 Lý do chọn đề tài	6
1.3 Ý nghĩa của dự án	6
1.4. Kết nối với xu hướng GenAI toàn cầu	6
2. Kiến trúc chi tiết	7
2.1. Tổng quan.....	7
2.2. Cấu trúc thư mục	7
2.3. Luồng hoạt động (Workflow)	8
2.4. Công nghệ sử dụng	9
2.5. Kiến trúc lớp (Layered Architecture).....	10
3. Dữ liệu (Data).....	12
3.1 Nguồn dữ liệu.....	12
3.2. Mẫu dữ liệu thực tế	12
3.3. Các trường dữ liệu	12
3.4. Quy trình thu thập dữ liệu	13
3.5. Tần suất và phạm vi thu thập	13
3.6. Đặc điểm dữ liệu.....	14
3.7. Mục tiêu sử dụng dữ liệu	14
4. Thuật toán & mô hình huấn luyện	14
4.1. Tổng quan.....	14
4.2. Thuật toán LightGBM (Light Gradient Boosting Machine).....	14
4.3. Cấu trúc mô hình huấn luyện.....	15
4.4. Quy trình huấn luyện mô hình	16
5. Giao diện AIRCARE	19
5.1. Chức năng chính	19
5.2. Dự đoán AQI trong 1, 3, 6 giờ tới	20
5.3. Cảnh báo khi người dùng nhập thông tin bệnh lý	21

6. Hướng phát triển	21
7. Tài liệu tham khảo.....	22

DỰ ÁN: AIRCARE

Dự đoán chất lượng không khí & cảnh báo sức khỏe cá nhân

1. Giới thiệu

1.1 Bối cảnh

Trong những năm gần đây, vấn đề ô nhiễm không khí đã trở thành một trong những thách thức môi trường lớn nhất trên toàn cầu. Theo báo cáo của Tổ chức Y tế Thế giới (WHO, 2024), ô nhiễm không khí là nguyên nhân khiến hơn 7 triệu người tử vong mỗi năm, chủ yếu do các bệnh về tim mạch, hô hấp và ung thư phổi.

Việt Nam, đặc biệt là các đô thị lớn như Hà Nội thường xuyên nằm trong nhóm các thành phố có chỉ số chất lượng không khí (AQI) ở mức “Kém” hoặc “Có hại cho sức khỏe”, đặc biệt vào mùa đông hoặc các thời điểm giao mùa.

Dự báo chất lượng không khí (AQI) tại Hà Nội			
ngày	Mức ô nhiễm		
thứ sáu, Th10 4	Không lành mạnh	154 AQI US	
thứ bảy, Th10 5	Không lành mạnh	159 AQI US	
chủ nhật, Th10 6	Không lành mạnh	166 AQI US	
Hôm nay	Không lành mạnh	168 AQI US	
thứ ba, Th10 8	Trung bình	86 AQI US	
thứ tư, Th10 9	Trung bình	91 AQI US	
thứ năm, Th10 10	Trung bình	93 AQI US	

Trong 4 ngày, từ 4/10 đến 7/10, Hà Nội liên tục ô nhiễm không khí ở mức nghiêm trọng

Nguồn: AQI

Song song với sự phát triển của công nghệ Internet of Things (IoT) và Trí tuệ nhân tạo (AI), các hệ thống cảm biến và nền tảng dữ liệu mở như OpenWeatherMap, AirVisual, Aqicn.org đã tạo điều kiện thuận lợi để người dùng có thể truy cập và theo dõi dữ liệu không khí theo thời gian thực. Tuy nhiên, phần lớn các hệ thống hiện nay chỉ dừng lại ở mức hiển thị dữ liệu hiện tại, chưa có khả năng dự đoán xu hướng ô nhiễm trong tương lai hay cá nhân hóa cảnh báo theo từng đối tượng người dùng.

Trong bối cảnh đó, dự án AIRCARE được đề xuất như một giải pháp ứng dụng Trí tuệ nhân tạo kết hợp Generative AI (GenAI Application) nhằm:

- Dự đoán chỉ số AQI trong ngắn hạn (1h, 3h, 6h tiếp theo) dựa trên các thông số môi trường thực tế (pm2_5, pm10, no2, co, o3, so2, temp, humidity, aqi).
- Sinh phản hồi thông minh và cảnh báo sức khỏe cá nhân hóa thông qua giao diện chatbot AI thân thiện, giúp người dùng hiểu rõ mức độ nguy hại và nhận được lời khuyên phù hợp với độ tuổi, thể trạng và tình trạng bệnh lý;
- Hướng tới mục tiêu nâng cao nhận thức cộng đồng về sức khỏe môi trường và hỗ trợ người dân ra quyết định (ví dụ: có nên ra ngoài, nên đeo khẩu trang, hoặc hạn chế vận động).

1.2 Lý do chọn đề tài

- Ô nhiễm không khí đang ngày càng nghiêm trọng, đặc biệt tại Việt Nam — đòi hỏi công cụ cảnh báo thông minh và trực quan.
- Người dân hiện nay chưa có giải pháp dự đoán AQI ngắn hạn, mà chỉ biết tình trạng hiện tại.
- Công nghệ Machine Learning và Generative AI đang phát triển mạnh, cho phép xây dựng các hệ thống dự đoán và tương tác ngôn ngữ tự nhiên hiệu quả với chi phí thấp.
- Việc kết hợp mô hình dự đoán với mô hình sinh phản hồi (Generative AI) giúp tạo ra một hệ thống thông minh hoàn chỉnh, không chỉ dự đoán mà còn hiểu và giao tiếp với người dùng.
- Dự án có thể mở rộng trong tương lai cho quy mô Smart City, tích hợp IoT hoặc bản đồ không khí.

1.3 Ý nghĩa của dự án

- Về mặt công nghệ:
 - + Ứng dụng AI lai (Hybrid AI): kết hợp giữa Machine Learning và Generative AI.
 - + Thực hành pipeline dữ liệu hoàn chỉnh: thu thập → xử lý → huấn luyện → triển khai.
 - + Triển khai hệ thống dự đoán trong thời gian thực (real-time inference).
- Về mặt xã hội:
 - + Cung cấp công cụ hữu ích giúp người dân theo dõi và dự đoán mức độ ô nhiễm không khí.
 - + Hỗ trợ người có bệnh lý hô hấp (hen suyễn, viêm phổi mạn tính, người cao tuổi) trong việc đưa ra quyết định an toàn khi ra ngoài.
 - + Góp phần nâng cao nhận thức cộng đồng về tác động của ô nhiễm không khí đến sức khỏe và hành vi sinh hoạt.

1.4. Kết nối với xu hướng GenAI toàn cầu

Năm 2024–2025 đánh dấu sự bùng nổ của Generative AI trong các lĩnh vực y tế, giáo dục, môi trường và chăm sóc sức khỏe cá nhân. Các hệ thống AI trợ lý môi trường

(Environmental AI Assistants) đang dần trở thành một phần của hạ tầng thông minh tại nhiều quốc gia.

Dự án AIRCARE hòa nhập vào xu hướng này khi:

- Ứng dụng mô hình ngôn ngữ lớn (LLM) để sinh ra các phản hồi tự nhiên và thân thiện;
- Kết hợp với dữ liệu cảm biến thực để đảm bảo phản hồi của chatbot có tính thực tiễn cao;
- Hướng tới trải nghiệm tương tác người dùng cá nhân hóa, thay vì chỉ là dashboard dữ liệu tĩnh.

2. Kiến trúc chi tiết

2.1. Tổng quan

Dự án AIRCARE — Chatbot Dự đoán AQI được thiết kế theo mô hình modular architecture, gồm 4 lớp chính:

- + Thu thập dữ liệu (Data Collection) – lấy dữ liệu thời tiết & không khí thực từ API OpenWeather.
- + Tiền xử lý & Huấn luyện mô hình (Model Training) – làm sạch dữ liệu, huấn luyện và lưu model dự đoán.
- + Triển khai API & Chatbot (Application Layer) – cung cấp giao diện giao tiếp và dự đoán theo thời gian thực.
- + Lưu trữ & kiểm thử (Testing & Storage) – quản lý dữ liệu, mô hình, và kiểm thử các endpoint.

2.2. Cấu trúc thư mục

AIRCARE/

```
|
|
| — .venv312/          # Môi trường ảo Python (tự động tạo khi dùng venv)
|   | — Scripts/       #Chứa file kích hoạt môi trường (activate)
|   |   — Lib/, Include/, etc.    # Các thư viện Python cài trong venv
|
|
| — data/              # Thư mục dữ liệu
|   | — raw/           # Dữ liệu gốc (chưa xử lý)
|   |   | — air_data.csv    # File dữ liệu AQI gốc (các thông số môi trường)
|   |   | — collector.log   # File log khi thu thập dữ liệu
```

```

|— models/                #Thư mục lưu các mô hình đã huấn luyện
| |— aqi_model_1h.pkl      #Mô hình dự đoán AQI 1 giờ tới
| |— aqi_model_3h.pkl      #Mô hình dự đoán AQI 3 giờ tới
| |— aqi_model_6h.pkl      #Mô hình dự đoán AQI 6 giờ tới
| |— feature_names_1h.pkl  #Danh sách đặc trưng tương ứng với model 1h
| |— feature_names_3h.pkl  #Danh sách đặc trưng tương ứng với model 3h
| |— feature_names_6h.pkl  #Danh sách đặc trưng tương ứng với model 6h
|— src/                   #Thư mục mã nguồn chính của dự án
| |— data/                #Chứa các script thu thập & xử lý dữ liệu
| | |— .env               #File lưu biến môi trường (API key, URL, v.v.)
| | |— collector.py       #Script tự động thu thập dữ liệu AQI từ web
| |
| |— models/              #Chứa các script huấn luyện mô hình
| | |— train_model.py     #Script huấn luyện và lưu các mô hình vào /models
| |— run/                 #Thư mục chạy ứng dụng chính
| | |— app.py             #File Streamlit chính — giao diện chatbot AQI
|— .gitignore             #Khai báo các file/thư mục không đẩy lên GitHub (.venv312)
|— README.md              #Mô tả tổng quan dự án, cách chạy, cấu trúc, tính năng
|— requirements.txt        #Danh sách thư viện cần

```

2.3. Luồng hoạt động (Workflow)

Bước 1 – Thu thập dữ liệu

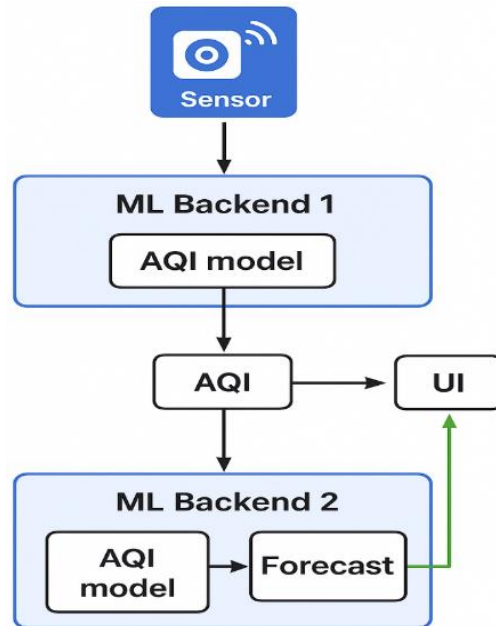
Người dùng chạy file collector.py

→ hệ thống gọi API OpenWeather

→ thu thập thông tin thời tiết và chỉ số AQI thực tế

→ lưu vào data/raw/air_data.csv.

Bước 2 – Tiền xử lý và huấn luyện mô hình



Chạy file `train_model.py` → dữ liệu được làm sạch, chia train/test → huấn luyện mô hình LightGBM (Light Gradient Boosting Machine) → xuất ra 3 model .pkl (dự đoán AQI sau 1h, 3h, 6h).

Bước 3 – Triển khai ứng dụng AI

- + Nhận đầu vào từ người dùng (vị trí, bệnh lý, giờ cần dự đoán).
- + Tải mô hình đã huấn luyện từ thư mục
- + Trả về kết quả dự đoán AQI, cùng lời khuyên sức khỏe phù hợp.

Bước 4 – Hiện thị kết quả và cảnh báo

- + Biểu đồ dự đoán AQI.
- + Cảnh báo sức khỏe theo từng mức AQI.
- + Chatbot hỗ trợ hỏi đáp bằng ngôn ngữ tự nhiên.

2.4. Công nghệ sử dụng

Hệ thống AIRCARE được xây dựng dựa trên các công nghệ và thư viện hiện đại trong lĩnh vực khoa học dữ liệu, học máy và trí tuệ nhân tạo:

- Python: Ngôn ngữ lập trình chính, dễ đọc, dễ mở rộng, phù hợp cho xử lý dữ liệu và xây dựng mô hình học máy.

+ Pandas và NumPy: Dùng để xử lý, làm sạch, và phân tích dữ liệu.

- + Scikit-learn và LightGBM: Hỗ trợ xây dựng và huấn luyện mô hình dự đoán AQI.
- + Joblib: Dùng để lưu trữ và tải lại các mô hình huấn luyện (.pkl).
- OpenWeatherMap API: Cung cấp dữ liệu thời tiết và chất lượng không khí theo thời gian thực, được sử dụng trong quá trình thu thập dữ liệu định kỳ 2 phút/lần.
- Generative AI (Large Language Model - LLM): Ứng dụng mô hình ngôn ngữ tự nhiên để tạo chatbot tương tác thân thiện, có khả năng hiểu và phản hồi thông minh các câu hỏi của người dùng về chất lượng không khí.

2.5. Kiến trúc lớp (Layered Architecture)

Hệ thống AIRCARE được thiết kế theo mô hình kiến trúc nhiều lớp (Layered Architecture), giúp phân tách rõ ràng các chức năng, tăng tính mở rộng, dễ bảo trì và thuận tiện khi triển khai. Cụ thể, hệ thống bao gồm 4 lớp chính như sau:

* Lớp Thu thập dữ liệu (Data Collection Layer)

- Chức năng: Tự động thu thập dữ liệu không khí thực tế từ các nguồn web API hoặc dữ liệu cảm biến định kỳ (mỗi 2 phút).

- Thực hiện tại: collector.py

- Nhiệm vụ:

- + Gọi API hoặc đọc dữ liệu từ cảm biến.
- + Chuẩn hóa, làm sạch dữ liệu đầu vào.
- + Lưu dữ liệu vào tệp data/raw/air_data.csv để phục vụ huấn luyện và dự đoán.

* Lớp Xử lý và huấn luyện mô hình (Processing & Model Layer)

+ Chức năng: Xử lý dữ liệu, trích xuất đặc trưng, huấn luyện mô hình dự đoán AQI.

+ Thực hiện tại: train_model.py

+ Thuật toán sử dụng: LightGBM Regressor (Gradient Boosting Decision Tree).

+ Nhiệm vụ:

- + Làm sạch dữ liệu và tạo các đặc trưng (feature engineering).
- + Huấn luyện 3 mô hình riêng biệt cho dự báo AQI sau 1h, 3h và 6h.
- + Lưu các mô hình đã huấn luyện vào thư mục models/ dưới dạng .pkl.

* Lớp Ứng dụng và Giao diện người dùng (Application/UI Layer)

- Chức năng: Hiển thị thông tin AQI hiện tại, dự đoán tương lai và hỗ trợ trò chuyện với người dùng.

- Thực hiện tại: app.py (xây dựng bằng Streamlit).

- Nhiệm vụ:

+ Tự động gọi lớp thu thập dữ liệu và lớp huấn luyện để cập nhật dữ liệu mới.

+ Giao diện chatbot hỗ trợ người dùng hỏi – đáp bằng ngôn ngữ tự nhiên (dựa trên SentenceTransformer).

+ Hiển thị biểu đồ, cảnh báo sức khỏe, và khuyến nghị theo từng mức AQI.

* Lớp Lưu trữ và Quản lý dữ liệu (Data Storage Layer)

- Chức năng: Lưu trữ dữ liệu và mô hình để phục vụ cho cả quá trình huấn luyện và dự đoán.

- Thực hiện tại:

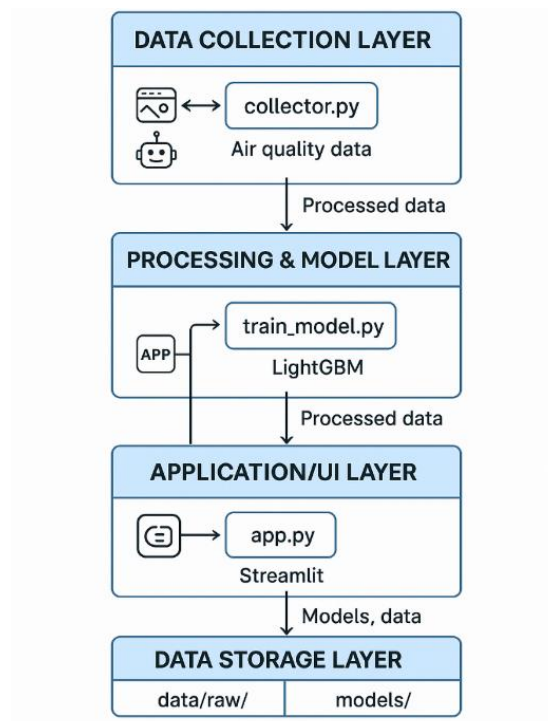
+ data/raw/: chứa dữ liệu gốc thu thập được.

+ models/: chứa các mô hình LightGBM đã huấn luyện và tệp danh sách đặc trưng.

- Nhiệm vụ:

+ Đảm bảo toàn vẹn dữ liệu.

+ Cung cấp dữ liệu cho lớp dự đoán và giao diện.



3. Dữ liệu (Data)

3.1 Nguồn dữ liệu

Dữ liệu được thu thập từ OpenWeatherMap API, một nền tảng cung cấp thông tin thời tiết và chất lượng không khí theo thời gian thực.

Mỗi lần người dùng chạy chương trình collector.py, hệ thống sẽ:

- Gửi yêu cầu đến API để xác định tọa độ thành phố (như Hà Nội).
- Gọi API Air Pollution và Weather để nhận các chỉ số môi trường.
- Gộp dữ liệu này và lưu vào file CSV cục bộ.

Dữ liệu được lưu tại: data/raw/air_data.csv

3.2. Mẫu dữ liệu thực tế

```
data > raw > air_data.csv > data
1 timestamp,pm2_5,pm10,no2,co,o3,so2,temp,humidity,aqi
2 2025-10-23 15:15:34,17.49,20.69,3.7,198.73,81.31,4.81,20.97,78,100
3 2025-10-23 15:17:35,17.49,20.69,3.7,198.73,81.31,4.81,20.97,78,100
4 2025-10-23 15:19:36,17.49,20.69,3.7,198.73,81.31,4.81,20.96,74,100
5 2025-10-23 15:21:38,17.49,20.69,3.7,198.73,81.31,4.81,20.98,75,100
6 2025-10-23 15:23:39,17.49,20.69,3.7,198.73,81.31,4.81,20.98,74,100
7 2025-10-23 15:25:40,17.49,20.69,3.7,198.73,81.31,4.81,20.98,74,100
8 2025-10-23 15:27:42,17.49,20.69,3.7,198.73,81.31,4.81,20.96,74,100
9 2025-10-23 15:29:43,17.49,20.69,3.7,198.73,81.31,4.81,20.95,74,100
```

Mỗi dòng tương ứng với một lần ghi nhận từ API tại thành phố Hà Nội.

3.3. Các trường dữ liệu

Trường	Kiểu dữ liệu	Đơn vị	Mô tả
timestamp	Datetime	YYYY-MM-DD HH:MM:SS	Thời điểm ghi nhận dữ liệu.
pm2_5	Float	$\mu\text{g}/\text{m}^3$	Nồng độ bụi mịn có kích thước nhỏ hơn 2.5 micromet. Gây hại trực tiếp cho phổi, tim và mạch máu.
pm10	Float	$\mu\text{g}/\text{m}^3$	Nồng độ bụi thô có kích thước nhỏ hơn 10 micromet.
no2	Float	$\mu\text{g}/\text{m}^3$	Nồng độ Nitơ Dioxid – khí thải từ phương tiện và nhà máy.
co	Float	$\mu\text{g}/\text{m}^3$	Nồng độ Carbon Monoxid – khí độc gây ngạt, chủ yếu do đốt cháy nhiên liệu.
o3	Float	$\mu\text{g}/\text{m}^3$	Nồng độ Ozon tầng thấp – có thể gây kích ứng hô hấp.

so2	Float	$\mu\text{g}/\text{m}^3$	Nồng độ Lưu huỳnh Dioxid – khí gây viêm phổi và đau họng khi hít lâu.
temp	Float	$^{\circ}\text{C}$	Nhiệt độ không khí hiện tại.
humidity	Integer	%	Độ ẩm tương đối trong không khí.
aqi	Integer	0–500 (chuẩn hóa từ thang 1–5)	Chỉ số chất lượng không khí. Giá trị càng cao $\rightarrow$ không khí càng ô nhiễm.

3.4. Quy trình thu thập dữ liệu

Quá trình thu thập được thực hiện theo chu trình tự động như sau:

- Khởi chạy lệnh: `python src/data/collector.py`

- Hệ thống gửi yêu cầu đến API:

https://api.openweathermap.org/data/2.5/air_pollution

<https://api.openweathermap.org/data/2.5/weather>

- Nhận phản hồi JSON, trích xuất các chỉ số cần thiết.

- Tạo dòng dữ liệu và thêm vào file CSV `data/raw/air_data.csv`.

The screenshot shows a code editor with a CSV file named `air_data.csv` open. The file contains a header row with the following fields: `timestamp, pm2_5, pm10, no2, co, o3, so2, temp, humidity, aqi`. Below the header, there is a data row with the following values: `2025-10-26 22:05:51, 70.41, 81.17, 8.17, 438.52, 21.97, 2.78, 24.98, 71, 200`. Below the code editor, there is a terminal window showing the output of the `python src/data/collector.py` command. The terminal output shows the following messages:
 - `Bắt đầu thu thập dữ liệu không khí liên tục...`
 - `Lần thứ 1: Đang lấy dữ liệu cho Hanoi...`
 - `Đã lưu dữ liệu vào data/raw/air_data.csv lúc 2025-10-26 22:05:51`

3.5. Tần suất và phạm vi thu thập

- Khu vực: Thành phố Hà Nội (mặc định), có thể mở rộng thêm nhiều thành phố khác bằng cách thay đổi biến `DEFAULT_CITY` trong `.env`.

- Tần suất gửi liệu cập nhật: Thời gian thực – thông qua OpenWeather API (độ trễ trung bình 2 phút).

3.6. Đặc điểm dữ liệu

- Kích thước mẫu ban đầu: Mỗi lần thu thập tạo ra một dòng dữ liệu mới, có thể tích lũy thành bộ dữ liệu lớn theo thời gian.
- Độ tin cậy: Nguồn dữ liệu chính thống, được cập nhật liên tục bởi OpenWeatherMap.
- Độ nhiễu: Thấp, tuy nhiên có thể phát sinh giá trị thiếu (NaN) khi API trả về lỗi hoặc mạng gián đoạn.
- Định dạng: CSV.

3.7. Mục tiêu sử dụng dữ liệu

Dữ liệu sau khi xử lý được dùng để:

- Huấn luyện mô hình dự đoán AQI theo thời gian.
- Đánh giá tương quan giữa các yếu tố ô nhiễm và thời tiết.
- Sinh phản hồi thông minh (Generative AI chatbot) để cảnh báo người dùng theo ngữ cảnh thực tế.

4. Thuật toán & mô hình huấn luyện

4.1. Tổng quan

Trong dự án AIRCARE – Chatbot Dự đoán AQI, nhóm sử dụng hai thành phần chính:

- Thuật toán học máy (Machine Learning): để dự đoán chỉ số chất lượng không khí (AQI) trong các khung thời gian tương lai.
- Mô hình sinh ngôn ngữ (Generative AI): để tạo phản hồi tự nhiên, đưa ra cảnh báo và lời khuyên cho người dùng.

Vì vậy, mô hình học cách dự đoán giá trị AQI (Air Quality Index) — một chỉ số liên tục, thường nằm trong khoảng $0 \rightarrow 500$, nên đây chính là bài toán hồi quy.

4.2. Thuật toán LightGBM (Light Gradient Boosting Machine)

- Khái niệm

LightGBM là một thuật toán Boosting dạng cây quyết định (Tree-Based Gradient Boosting), được phát triển bởi Microsoft.

Thuật toán hoạt động bằng cách xây dựng nhiều cây yếu (weak learners) theo chuỗi, trong đó mỗi cây mới học từ sai số (residual) của các cây trước đó.

- Nguyên lý hoạt động

+ Mô hình khởi đầu bằng một dự đoán đơn giản (ví dụ: trung bình của toàn bộ dữ liệu).

- + Tính toán sai số (residual) giữa dự đoán và giá trị thật.
- + Xây dựng cây quyết định mới để dự đoán phần sai số đó.
- + Cập nhật mô hình bằng cách cộng dự đoán của cây mới, có trọng số học được qua gradient.
- + Lặp lại cho đến khi hội tụ hoặc đạt giới hạn số vòng lặp.

4.3. Cấu trúc mô hình huấn luyện

- Mã nguồn huấn luyện được cài đặt trong file: src/models/train_model.py

The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'src', 'models', and 'data'. The code editor shows the content of 'train_model.py'.

```

1  import os
2  import pandas as pd
3  import numpy as np
4  import lightgbm as lgb
5  from sklearn.model_selection import train_test_split
6  from sklearn.metrics import mean_absolute_error, r2_score
7  import joblib
8
9  # =====
10 # 1. CẤU HÌNH
11 # =====
12 DATA_PATH = "data/raw/air_data.csv"
13 MODEL_DIR = "models"
14 HORIZONS = [1, 3, 6] # Dự báo 1h, 3h, 6h tới
15
16 # =====
17 # 2. NẠP DỮ LIỆU
18 # =====
19 if not os.path.exists(DATA_PATH):
20     raise FileNotFoundError(f"Không tìm thấy file dữ liệu tại {DATA_PATH}")
21
22 data = pd.read_csv(DATA_PATH, parse_dates=["timestamp"])
23 data = data.drop_duplicates(subset="timestamp").sort_values("timestamp")
24 data = data.dropna(subset=["aqi"])
25
26 print(f"✓ Nạp dữ liệu: {len(data)} dòng, {data['timestamp'].min()} → {data['timestamp'].max()}")
27
28 # =====
29 # 3. FEATURE ENGINEERING
30 # =====
31 for col in ["pm2_5", "pm10", "co", "no2", "o3", "so2", "aqi"]:
32     for lag in range(1, 4): # tạo độ trễ 1-3 giờ
33         data[f"{col}_lag{lag}"] = data[col].shift(lag)
34
35 data["aqi_rolling3"] = data["aqi"].rolling(window=3).mean()
36 data["hour"] = data["timestamp"].dt.hour
37 data["weekday"] = data["timestamp"].dt.weekday
  
```

- Mô hình được huấn luyện tự động cho 3 mốc thời gian dự báo:
 - + AQI sau 1 giờ
 - + AQI sau 3 giờ
 - + AQI sau 6 giờ
- Mỗi mô hình được lưu lại dạng .pkl trong thư mục models/.

The screenshot shows a file explorer view of the 'models' directory. It contains several files with names indicating the horizon and the type of data stored.

```

models
├── aqi_model_1h.pkl
├── aqi_model_3h.pkl
├── aqi_model_6h.pkl
├── feature_names_1h.pkl
├── feature_names_3h.pkl
├── feature_names_6h.pkl

```

4.4. Quy trình huấn luyện mô hình

(1) Nạp và xử lý dữ liệu

- Dữ liệu thực tế từ OpenWeatherMap được lưu tại data/raw/air_data.csv.
- Sau khi đọc, dữ liệu được sắp xếp theo thời gian (timestamp), loại bỏ bản ghi trùng lặp và giá trị thiếu.

```
data = pd.read_csv(DATA_PATH, parse_dates=["timestamp"])
data = data.drop_duplicates(subset="timestamp").sort_values("timestamp")
data = data.dropna(subset=["aqi"])
```

+ parse_dates=["timestamp"]: Chuyển cột thời gian sang kiểu *datetime* giúp thuận tiện cho việc trích xuất đặc trưng thời gian.

+ drop_duplicates(subset="timestamp"): Xóa các bản ghi trùng thời gian nhằm tránh sai lệch khi huấn luyện.

+ dropna(subset=["aqi"]): Loại bỏ dòng bị thiếu giá trị AQI để đảm bảo đầu ra huấn luyện chính xác.

(2) Feature Engineering – Xây dựng đặc trưng

- Tạo độ trễ (lag features) cho các chỉ số như pm2_5, pm10, no2, co, o3, so2, aqi để mô hình học được xu hướng biến động.
- Thêm đặc trưng thời gian như giờ trong ngày (hour), ngày trong tuần (weekday), tháng (month), và biểu diễn tuần hoàn (sin, cos) của giờ để mô hình học chu kỳ ngày–đêm.

```
for col in ["pm2_5", "pm10", "co", "no2", "o3", "so2", "aqi"]:
    for lag in range(1, 4): # tạo độ trễ 1-3 giờ
        data[f"{col}_lag{lag}"] = data[col].shift(lag)

data["aqi_rolling3"] = data["aqi"].rolling(window=3).mean()
data["hour"] = data["timestamp"].dt.hour
data["weekday"] = data["timestamp"].dt.weekday
data["month"] = data["timestamp"].dt.month
data["hour_sin"] = np.sin(2 * np.pi * data["hour"] / 24)
data["hour_cos"] = np.cos(2 * np.pi * data["hour"] / 24)
```

+ Dòng lệnh data[f"{col}_lag{lag}"] = data[col].shift(lag) tạo ra các cột mới biểu diễn giá trị các chỉ số (PM2.5, NO₂, AQI,...) của 1–3 giờ trước.

+ Tạo đặc trưng thống kê: aqi_rolling3 là trung bình trượt của AQI trong 3 giờ, giúp giảm nhiễu và phản ánh xu hướng ngắn hạn.

+ Đặc trưng thời gian:

- Các cột hour, weekday, month thể hiện chu kỳ giờ, ngày, tháng.
- Các cột hour_sin, hour_cos giúp mô hình hiểu chu kỳ tuần hoàn của ngày – đêm, tránh việc mô hình xem 23h và 0h là hai mốc rời rạc.

(3) Tạo dữ liệu mục tiêu tương lai

Để huấn luyện mô hình dự đoán AQI trong tương lai, cột mục tiêu được tạo bằng cách dịch (shift) chỉ số AQI về sau:

```
def train_for_horizon(horizon: int):  
    df = data.copy()  
    df["aqi_future"] = df["aqi"].shift(-horizon)  
    df = df.dropna()
```

Dòng lệnh này tạo ra biến mục tiêu aqi_future, đại diện cho giá trị AQI trong tương lai tương ứng với khoảng thời gian horizon (1, 3 hoặc 6 giờ).

Mô hình LightGBM sau đó được huấn luyện để dự đoán giá trị aqi_future dựa trên các đặc trưng hiện tại và quá khứ (như PM2.5, PM10, NO₂, CO, O₃, SO₂, nhiệt độ, độ ẩm, v.v.).

Nhờ đó, hệ thống có thể dự báo xu hướng biến động AQI trong ngắn hạn (1h, 3h, 6h), hỗ trợ người dùng chủ động hơn trong việc bảo vệ sức khỏe và lập kế hoạch di chuyển.

(4) Chia dữ liệu train/test

Không dùng xáo trộn ngẫu nhiên (shuffle=False) để đảm bảo thứ tự thời gian.

```
x_train, x_test, y_train, y_test = train_test_split(  
    x, y, test_size=0.2, shuffle=False  
)
```

Dữ liệu được chia thành 80% cho train và 20% cho test bằng hàm train_test_split().

Để đảm bảo tính liên tục theo thời gian, tham số shuffle=False được dùng – nghĩa là dữ liệu không bị xáo trộn ngẫu nhiên. Cách này giúp mô hình học đúng xu hướng thời gian thực, phù hợp với bài toán dự đoán chuỗi thời gian AQI.

(5) Huấn luyện mô hình LightGBM

Mô hình LightGBM Regressor được chọn vì khả năng xử lý dữ liệu lớn và tốc độ huấn luyện nhanh.

- Các tham số được thiết lập gồm:
- + objective='regression': bài toán dự báo giá trị liên tục (AQI).
- + metric='mae': dùng sai số tuyệt đối trung bình để đánh giá.
- + boosting_type='gbdt': thuật toán boosting dạng gradient tree.
- + learning_rate=0.05: tốc độ học vừa phải để tránh quá khớp.
- + num_leaves=31: số lượng lá trên cây quyết định độ phức tạp mô hình.
- + feature_fraction và bagging_fraction: giúp giảm overfitting bằng cách chỉ chọn ngẫu nhiên một phần đặc trưng và dữ liệu mỗi lần huấn luyện.

```
params = {
    "objective": "regression",
    "metric": "mae",
    "boosting_type": "gbdt",
    "learning_rate": 0.05,
    "num_leaves": 31,
    "feature_fraction": 0.8,
    "bagging_fraction": 0.8,
    "bagging_freq": 5,
    "seed": 42,
    "verbose": -1,
}
```

- Mô hình được huấn luyện với early stopping để tránh overfitting:

```
model = lgb.train(
    params=params,
    train_set=train_data,
    valid_sets=[val_data],
    num_boost_round=2000,
    callbacks=[lgb.early_stopping(stopping_rounds=100, verbose=False)],
)
```

Mô hình được huấn luyện với early stopping, nghĩa là nếu sai số trên tập kiểm định không giảm sau 100 vòng lặp, quá trình huấn luyện sẽ dừng lại sớm.

Điều này giúp mô hình không học quá kỹ dữ liệu huấn luyện (overfitting) và tổng quát tốt hơn khi dự đoán các giá trị AQI trong tương lai.

(6) Đánh giá mô hình

Hai chỉ số chính:

- MAE (Mean Absolute Error): sai số tuyệt đối trung bình
- R² (Coefficient of Determination): mức độ giải thích phương sai

```
PS C:\Users\admin\aircare> .\.venv312\Scripts\Activate.ps1
(.venv312) PS C:\Users\admin\aircare> python src/models/train_model.py
✓ Nạp dữ liệu: 513 dòng, 2025-10-23 15:15:34 → 2025-10-26 00:08:39

🚀 Bắt đầu huấn luyện mô hình dự báo AQI (1h, 3h, 6h)...

✓ AQI +1h → MAE: 0.516 | R²: -0.010 | 📁 aqi_model_1h.pkl
✓ AQI +3h → MAE: 0.556 | R²: -0.010 | 📁 aqi_model_3h.pkl
✓ AQI +6h → MAE: 0.550 | R²: -0.011 | 📁 aqi_model_6h.pkl

=====
❑ HIỆU SUẤT CÁC MÔ HÌNH DỰ BÁO AQI
=====
🕒 1h → MAE: 0.516 | R²: -0.010
🕒 3h → MAE: 0.556 | R²: -0.010
🕒 6h → MAE: 0.550 | R²: -0.011
=====
✓ Huấn luyện hoàn tất.
```

Các giá trị MAE (Mean Absolute Error) rất nhỏ — chỉ dao động trong khoảng 0.51 đến 0.55, cho thấy sai số trung bình giữa giá trị dự đoán và giá trị thực tế của mô hình là thấp.

Điều này chứng tỏ mô hình LightGBM đã học được tốt mối quan hệ giữa các thông số môi trường (PM2.5, PM10, NO₂, CO, O₃, SO₂, nhiệt độ, độ ẩm...) và chỉ số AQI tương ứng.

Tuy nhiên, ta cũng nhận thấy:

- + Giá trị R² (hệ số xác định) hơi âm (≈ -0.01), điều này cho thấy mô hình chưa khái quát tốt hoặc dữ liệu còn quá ít / chưa đủ đa dạng để mô hình học được xu hướng thực.

- + Do dữ liệu thử nghiệm ngắn hạn (≈ 500 mẫu), việc chia train/test có thể làm giảm chất lượng đánh giá.

(7) Lưu mô hình

```
joblib.dump(model, model_path)
joblib.dump(features, os.path.join(MODEL_DIR, f"feature_names_{horizon}h.pkl"))
```

Sau khi huấn luyện, mô hình LightGBM và danh sách đặc trưng được lưu lại bằng thư viện joblib để phục vụ cho việc dự đoán sau này:

- + aqi_model_{horizon}h.pkl: chứa mô hình đã huấn luyện cho từng mốc dự báo (1h, 3h, 6h).

- + feature_names_{horizon}h.pkl: chứa danh sách các đặc trưng đầu vào tương ứng.

Việc lưu song song mô hình và đặc trưng giúp đảm bảo khi tải lại để dự đoán, dữ liệu đầu vào có cấu trúc thống nhất với giai đoạn huấn luyện.

5. Giao diện AIRCARE

5.1. Chức năng chính

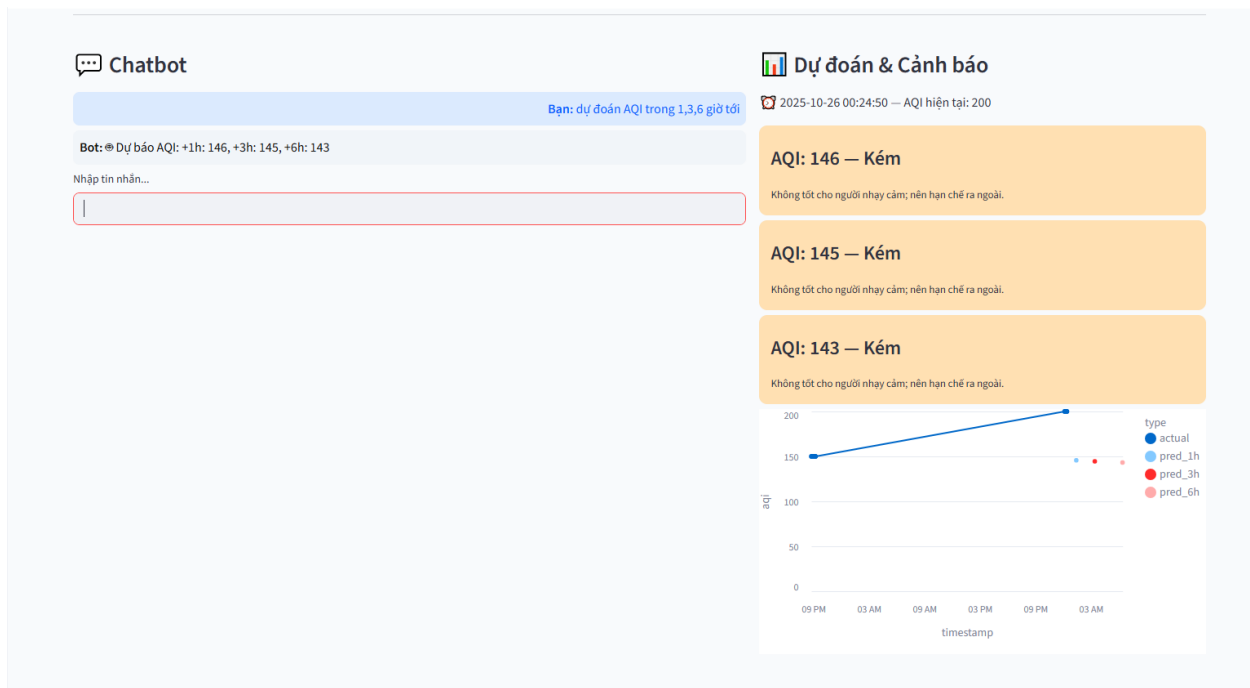
- Chatbot thông minh: nhận lệnh ngữ nghĩa như “Dự đoán AQI”, “Tôi nên ra ngoài không?”
- Tự động cập nhật dữ liệu: collector.py chạy nền mỗi 2 phút.

- Tự huấn luyện model: train_model khởi chạy nền khi có dữ liệu mới.
- Biểu đồ dự báo AQI: minh họa trực quan biến động trong 6 giờ tới.
- Cảnh báo sức khỏe: tùy theo AQI & bệnh lý người dùng nhập.

The screenshot shows the AIRCARE Chatbot interface. On the left, there's a sidebar with 'Thông tin người dùng' (User Information) including 'Tuổi' (Age) set to 25 and 'Bệnh lý (nếu có)' (If any disease) with a text input field containing 'VD: hen suyễn, viêm xoang, tim mạch...'. Below this is a 'Dự đoán nhanh' (Quick Prediction) button. The main area has the title 'AIRCARE — Chatbot Dự đoán AQI' and a subtitle 'Hỏi về chất lượng không khí hoặc dự đoán AQI trong 1h, 3h, 6h tới.' (Ask about air quality or predict AQI in 1h, 3h, 6h). There's a 'Chatbot' section with a 'Nhập tin nhắn...' (Enter message...) input field. On the right, there's a 'Dự đoán & Cảnh báo' (Prediction & Warning) section showing a timestamp '2025-10-26 18:40:26' and 'AQI hiện tại: 200'. Below this is a button that says 'Nhấn Dự đoán nhanh hoặc hỏi chatbot để xem dự báo.' (Click Quick Prediction or ask chatbot to see the forecast).

5.2. Dự đoán AQI trong 1, 3, 6 giờ tới

- Nếu người dùng muốn xem biểu đồ AQI dự đoán



- Nếu người dùng muốn xem dữ liệu AQI được dự đoán

Chatbot

Bạn: dự đoán AQI trong 1,3,6 giờ tới

Bot: @ Dự báo AQI: +1h: 146, +3h: 145, +6h: 143

Nhập tin nhắn...

Dự đoán & Cảnh báo

2025-10-26 00:24:50 — AQI hiện tại: 200

AQI: 146 — Kém

Không tốt cho người nhạy cảm; nên hạn chế ra ngoài.

AQI: 145 — Kém

Không tốt cho người nhạy cảm; nên hạn chế ra ngoài.

AQI: 143 — Kém

Không tốt cho người nhạy cảm; nên hạn chế ra ngoài.

	timestamp	aqi	type
19	2025-10-26 00:18:44	200	actual
20	2025-10-26 00:18:45	200	actual
21	2025-10-26 00:20:47	200	actual
22	2025-10-26 00:22:48	200	actual
23	2025-10-26 00:24:50	200	actual
24	2025-10-26 01:24:50	145.6794	pred_1h
25	2025-10-26 03:24:50	144.5217	pred_3h
26	2025-10-26 06:24:50	143.1855	pred_6h

5.3. Cảnh báo khi người dùng nhập thông tin bệnh lý

Thông tin người dùng

Tuổi

25

Bệnh lý (nếu có)

xoang mũi

Dự đoán nhanh

Chatbot

Bạn: dự đoán AQI trong 1,3,6 giờ tới

Bot: @ Dự báo AQI: +1h: 146, +3h: 145, +6h: 143

Bạn: người bị xoang mũi có nên ra đường

Bot: ⚠ Không khí ô nhiễm, hạn chế ra ngoài. Vì bạn có bệnh 'xoang mũi', nên chú ý hơn khi AQI >100.

Nhập tin nhắn...

Dự đoán & Cảnh báo

2025-10-26 00:24:50 — AQI hiện tại: 200

AQI: 146 — Kém

Không tốt cho người nhạy cảm; nên hạn chế ra ngoài.

AQI: 145 — Kém

Không tốt cho người nhạy cảm; nên hạn chế ra ngoài.

AQI: 143 — Kém

Không tốt cho người nhạy cảm; nên hạn chế ra ngoài.

	timestamp	aqi	type
19	2025-10-26 00:18:44	200	actual
20	2025-10-26 00:18:45	200	actual
21	2025-10-26 00:20:47	200	actual
22	2025-10-26 00:22:48	200	actual
23	2025-10-26 00:24:50	200	actual
24	2025-10-26 01:24:50	145.6794	pred_1h
25	2025-10-26 03:24:50	144.5217	pred_3h
26	2025-10-26 06:24:50	143.1855	pred_6h

6. Hướng phát triển

- Mở rộng nhiều thành phố (HCMC, Đà Nẵng, Hải Phòng...)
- Tích hợp cảm biến IoT đo bụi thực tế
- Triển khai trên AWS SageMaker / Streamlit Cloud

- Kết nối LLM lớn hơn (GPT, Claude) cho chatbot thông minh hơn
- Phát triển ứng dụng di động (Flutter)

7. Tài liệu tham khảo

Mã nguồn dự án GitHub: https://github.com/agnessktt/HTTM_NHOM8

[1] OpenWeatherMap. *API Documentation*. Truy cập từ: <https://openweathermap.org/api>

[2] World Health Organization (WHO). *Report on Air Pollution 2024*. Geneva: WHO, 2024.

[3] Microsoft. *LightGBM Official Guide*. Truy cập từ: <https://lightgbm.readthedocs.io>

[4] Hugging Face. *SentenceTransformer* — “*paraphrase-multilingual-MiniLM-L12-v2*”.
<https://www.sbert.net>

[5] Altair Developers. *Altair Visualization Library*. Truy cập từ: <https://altair-viz.github.io>